ጨዋታት ዘግዕዝ

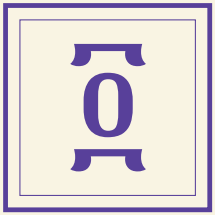
Large language models

Weekend 1: Representation and the transformer

Carlos Cotrini
AI in Industry, ETH Zurich
Friday 16 October 2026 ፳፩ጽ፮

The five Ge’ez games

	ሥርዓት **Structure**	We can extract structure from unfamiliar text without knowing a single word.
	ምሳሌ **Imitation**	By spotting patterns, we can compose new sentences in a language we don’t understand.
	ትንቢት **Prediction**	A language model picks the next word by probabilities conditioned on what came before.
	ምልክት **Representation**	A prefix becomes an embedding, and the embedding gives the chances of the next word.
	ቅንብር **Transformer**	An LLM computes the chances, draws a word, appends it, and repeats.



Section 4

Representation

Happy or unhappy?

The package was **good** and **fine**, so I am ___

The package was **late** and **broken**, so I am ___

Happy or unhappy?

The package was **good** and **fine**, so I am ___ → happy

The package was **late** and **broken**, so I am ___ → unhappy

Happy or unhappy?

The package was **good** and **fine**, so I am ___ → happy

The package was **late** and **broken**, so I am ___ → unhappy

You looked at a few telling words.

Happy or unhappy?

The package was **good** and **fine**, so I am ___ → happy

The package was **late** and **broken**, so I am ___ → unhappy



You looked at a few telling words.



A computer can do the same, if the words become numbers.

A corpus of reviews

Happy	Unhappy
The package was good and fine, so I am **happy**.	The package was late and broken, so I am **unhappy**.
The package was fine and the box was great, so I am **happy**.	The package was damaged and late, so I am **unhappy**.
The package was good, fine and great, so I am **happy**.	The package was fine but late and broken, so I am **unhappy**.

A corpus of reviews

Happy	Unhappy
The package was good and fine, so I am **happy**.	The package was late and broken, so I am **unhappy**.
The package was fine and the box was great, so I am **happy**.	The package was damaged and late, so I am **unhappy**.
The package was good, fine and great, so I am **happy**.	The package was fine but late and broken, so I am **unhappy**.



*Happy reviews say **good**, **fine**, **great**. Unhappy ones say **late**, **broken**, **damaged**.*

From a prefix to a guess

Prefix

The words so far

The package was **late**
and **broken**, so I am

From a prefix to a guess

Prefix

The words so far

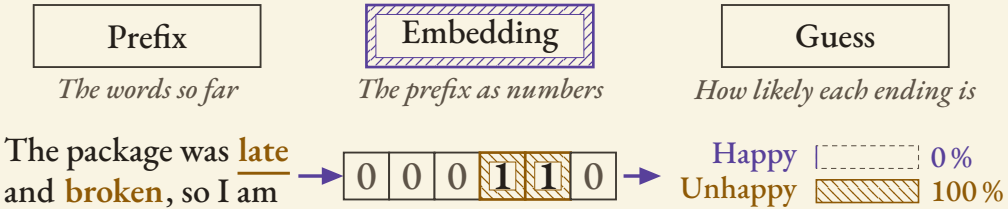
Embedding

The prefix as numbers

The package was **late** and **broken**, so I am →



From a prefix to a guess



Count the words

The package was late and broken, so I am

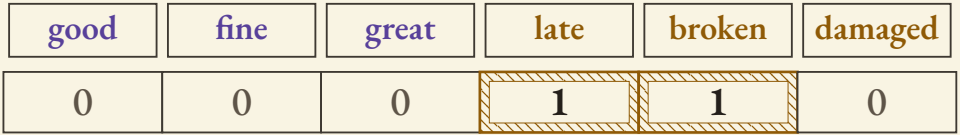
The dictionary: three happy words and three unhappy words

good	fine	great	late	broken	damaged

Count the words

The package was late and broken, so I am

The dictionary: three happy words and three unhappy words

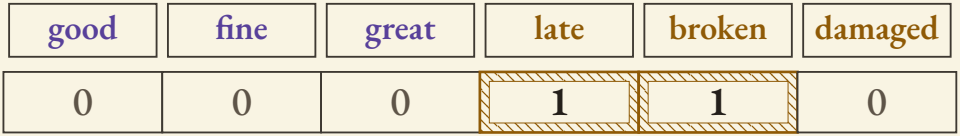


Every other word is ignored.

Count the words

The package was late and broken, so I am

The dictionary: three happy words and three unhappy words

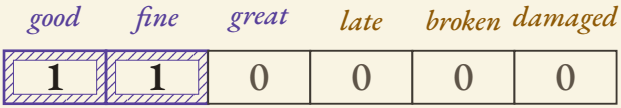


Every other word is ignored.

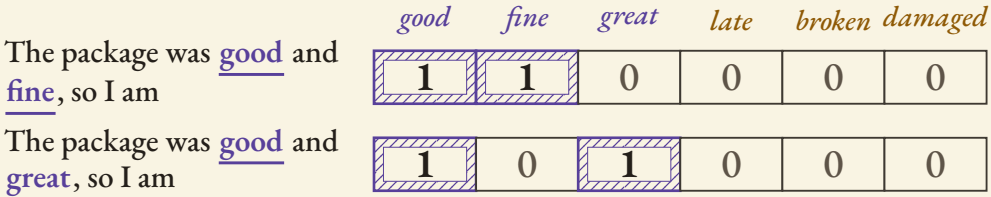
*This row of counts is the **embedding** of the prefix.*

Every prefix becomes a row

The package was good and fine, so I am

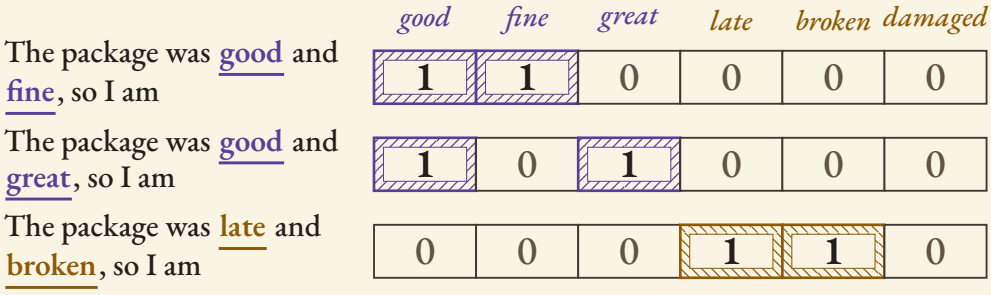


Every prefix becomes a row



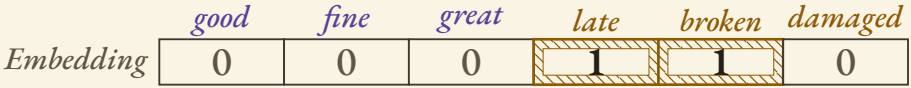
Similar reviews get similar rows.

Every prefix becomes a row



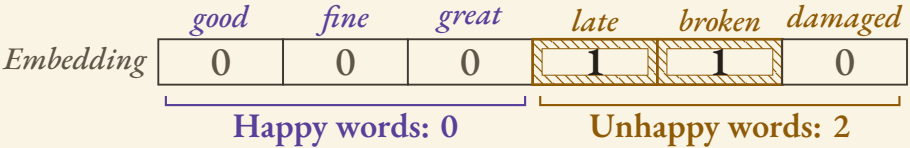
From counts to a guess

The package was **late** and **broken**, so I am



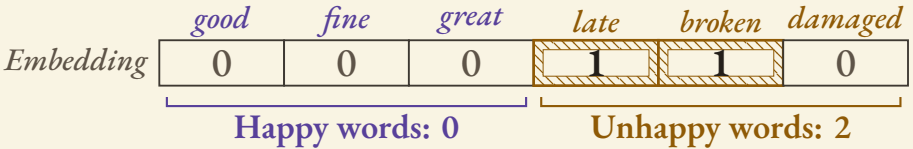
From counts to a guess

The package was **late** and **broken**, so I am



From counts to a guess

The package was **late** and **broken**, so I am

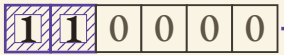


The whole pipeline

Prefix

The package was good and fine, so I am

Embedding

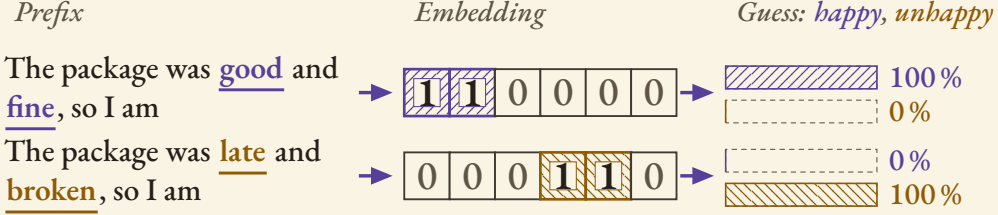


Guess: happy, unhappy



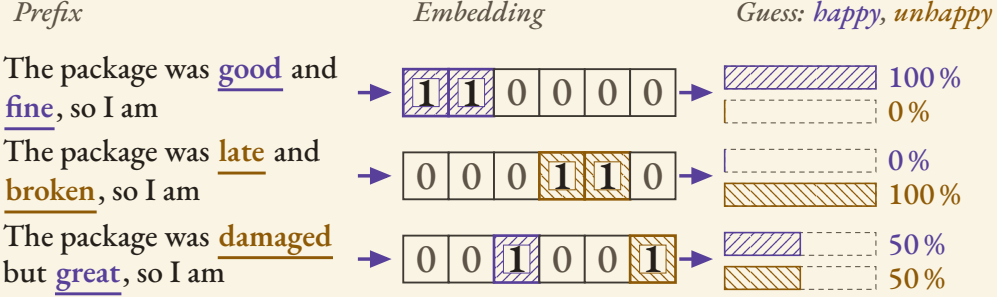
The cells follow the dictionary: good, fine, great, late, broken, damaged.

The whole pipeline



*The cells follow the dictionary: **good**, **fine**, **great**, **late**, **broken**, **damaged**.*

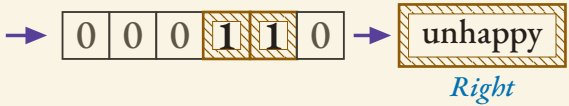
The whole pipeline



*The cells follow the dictionary: **good**, **fine**, **great**, **late**, **broken**, **damaged**.*

What counting misses

The package was **late** and **broken**, so I am *unhappy*



What counting misses

The package was late and broken, so I am *unhappy*



Right

The package was not late and not broken, so I am *happy*



Wrong

What counting misses

The package was late and broken, so I am *unhappy*



Right

The package was not late and not broken, so I am *happy*



Wrong

The dictionary has no slot for the word not.

What counting misses

The package was late and broken, so I am *unhappy*



Right

The package was not late and not broken, so I am *happy*



Wrong

The dictionary has no slot for the word not.

Counting forgets how the words combine.

What an embedding is



- **Embedding**: a text written as a row of numbers

What an embedding is



- **Embedding**: a text written as a row of numbers
- Here, one count for each word of a dictionary

What an embedding is

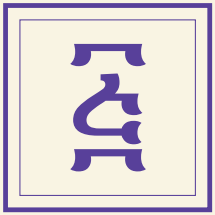


- **Embedding**: a text written as a row of numbers
- Here, one count for each word of a dictionary
- Happy words divided by all words gives the odds of a happy ending

What an embedding is



- **Embedding**: a text written as a row of numbers
- Here, one count for each word of a dictionary
- Happy words divided by all words gives the odds of a happy ending
- Real ones have thousands of numbers, learned by the model



Section 5

Transformer

A language model is a table

Prefix	*Alice*	*Bob*	*plays*	*sings*	*chess*	*golf*	*tennis*	*END*
START	0.6	0.4						
Alice			0.67	0.33				
Bob			1.0					
Alice plays						0.5	0.5	
Alice sings								1.0
Bob plays					0.5		0.5	
Alice plays golf								1.0
Alice plays tennis								1.0
Bob plays chess								1.0
Bob plays tennis								1.0

A language model is a table

Prefix	*Alice*	*Bob*	*plays*	*sings*	*chess*	*golf*	*tennis*	*END*
START	0.6	0.4						
Alice			0.67	0.33				
Bob			1.0					
Alice plays						0.5	0.5	
Alice sings								1.0
Bob plays					0.5		0.5	
Alice plays golf								1.0
Alice plays tennis								1.0
Bob plays chess								1.0
Bob plays tennis								1.0



One row for each prefix.

A language model is a table

Prefix	*Alice*	*Bob*	*plays*	*sings*	*chess*	*golf*	*tennis*	*END*
START	0.6	0.4						
Alice			0.67	0.33				
Bob			1.0					
Alice plays						0.5	0.5	
Alice sings								1.0
Bob plays					0.5		0.5	
Alice plays golf								1.0
Alice plays tennis								1.0
Bob plays chess								1.0
Bob plays tennis								1.0



One row for each prefix.



A row holds the chance of each next word.

A row is a probability distribution

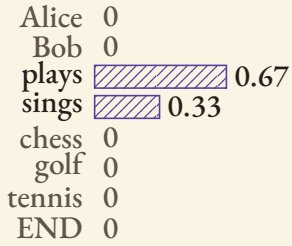
The row of the prefix **Alice**

Alice	*Bob*	*plays*	*sings*	*chess*	*golf*	*tennis*	*END*
0	0	**0.67**	**0.33**	0	0	0	0

A row is a probability distribution

The row of the prefix **Alice**

Alice	*Bob*	*plays*	*sings*	*chess*	*golf*	*tennis*	*END*
0	0	**0.67**	**0.33**	0	0	0	0

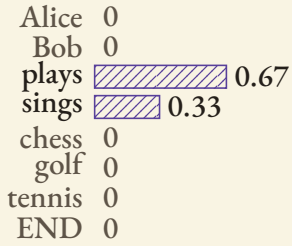


The same numbers, drawn as bars.

A row is a probability distribution

The row of the prefix **Alice**

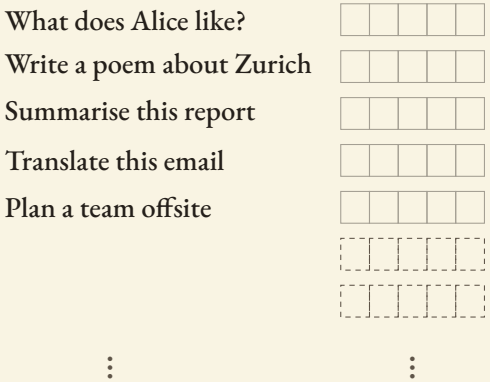
Alice	*Bob*	*plays*	*sings*	*chess*	*golf*	*tennis*	*END*
0	0	**0.67**	**0.33**	0	0	0	0



The same numbers, drawn as bars.

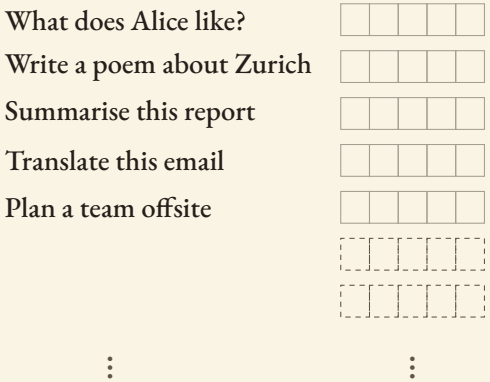
*The shares add up to 1: a **probability distribution**.*

The real table is too big



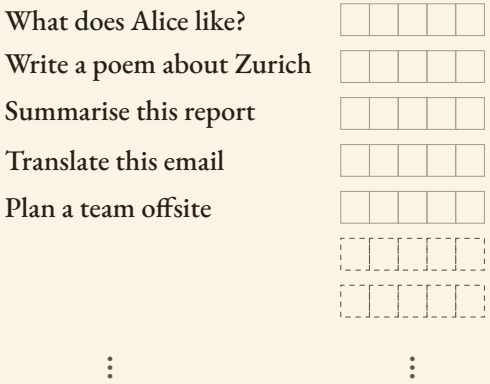
- One row for every possible prompt

The real table is too big



- One row for every possible prompt
- More rows than atoms in the universe

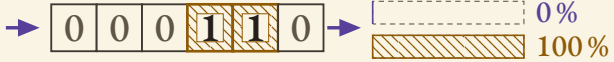
The real table is too big



- One row for every possible prompt
- More rows than atoms in the universe
- Too big for any computer

Compute each row when it is needed

The package was late and broken, so I am

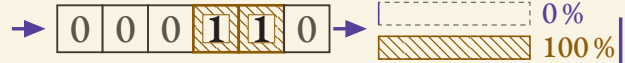


Prefix	*happy*	*unhappy*
The package was good and fine, so I am		
The package was late and broken, so I am		
The package was damaged but great, so I am		

⋮

Compute each row when it is needed

The package was late and broken, so I am

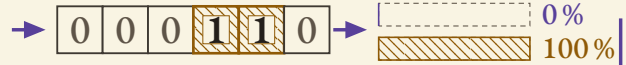


Prefix	*happy*	*unhappy*
The package was good and fine, so I am		
The package was late and broken, so I am	0 %	100 %
The package was damaged but great, so I am		

⋮

Compute each row when it is needed

The package was **late** and **broken**, so I am



Prefix	*happy*	*unhappy*
The package was good and fine, so I am	100 %	0 %
The package was late and broken, so I am	0 %	100 %
The package was damaged but great, so I am	50 %	50 %

⋮

The table is never stored. Each row is computed from its prefix.

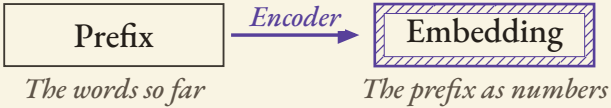
The recipe: encoder and decoder

Prefix

The words so far

Alice plays

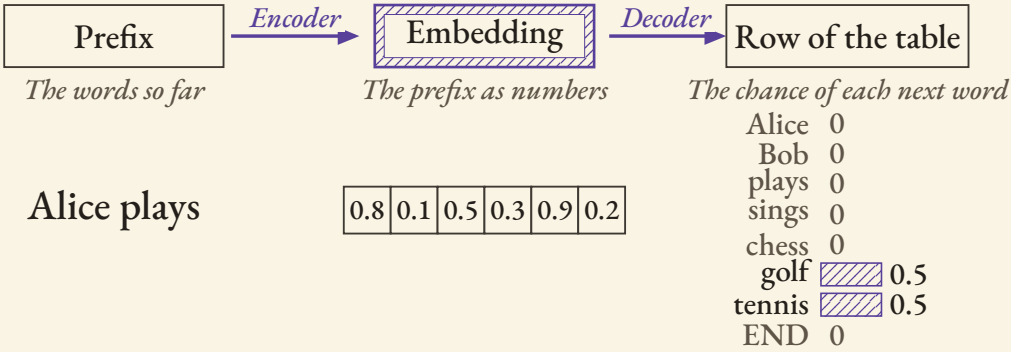
The recipe: encoder and decoder



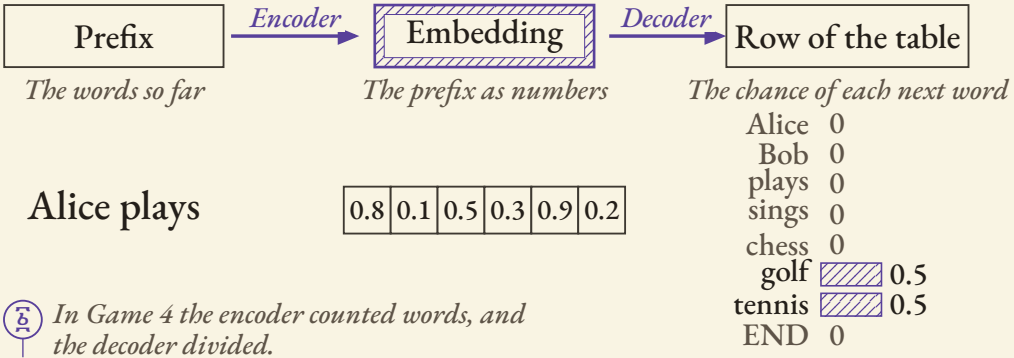
Alice plays

0.8	0.1	0.5	0.3	0.9	0.2

The recipe: encoder and decoder

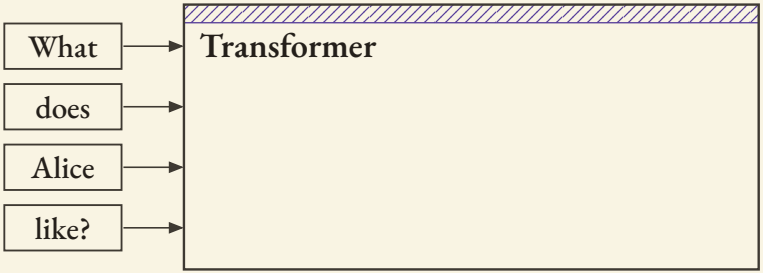


The recipe: encoder and decoder

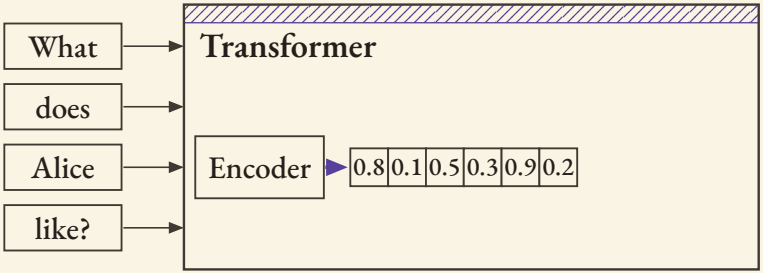


In Game 4 the encoder counted words, and the decoder divided.

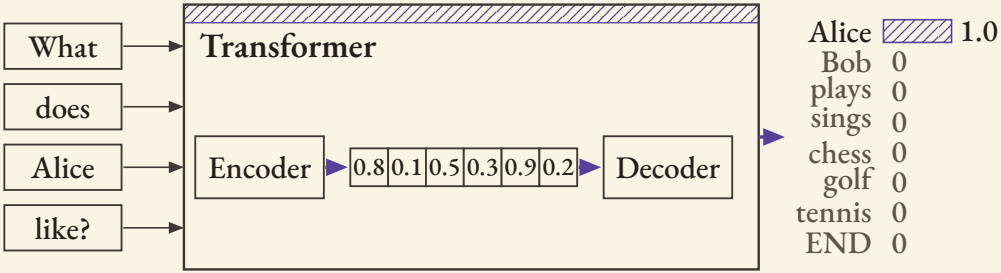
A transformer does both steps



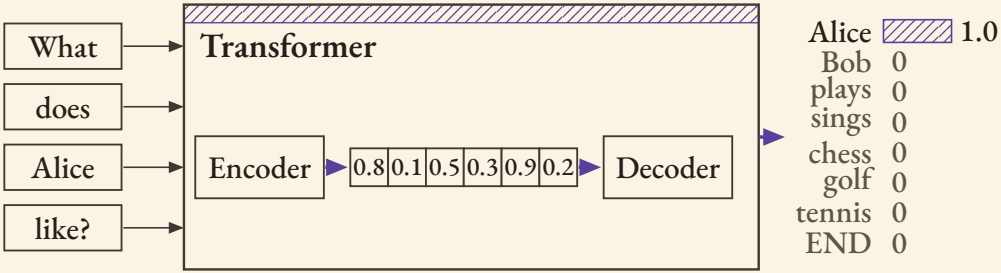
A transformer does both steps



A transformer does both steps

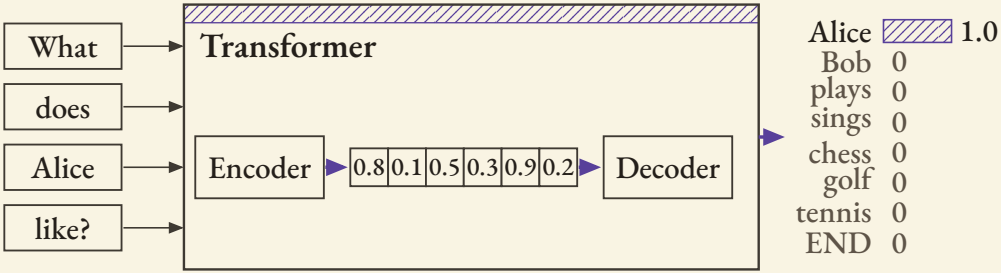


A transformer does both steps



It reads each word together with every other word.

A transformer does both steps



It reads each word together with every other word.

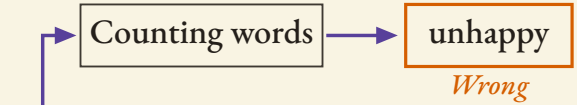
Its billions of numbers are learned from text.

A transformer reads how the words combine

The package was not
late and not broken, so
I am ___

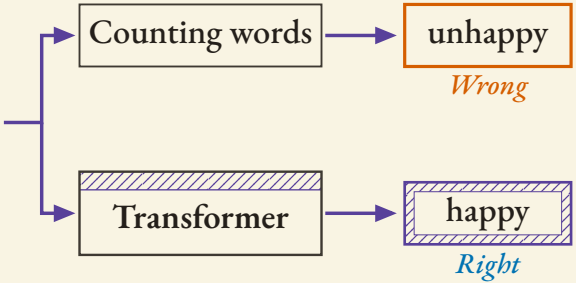
A transformer reads how the words combine

The package was not late and not broken, so I am ___



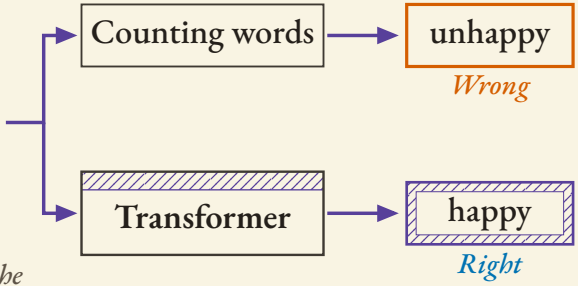
A transformer reads how the words combine

The package was not late and not broken, so I am ___



A transformer reads how the words combine

The package was not late and not broken, so I am ___



It reads not together with the words it changes.

From our toys to a large language model

	Our toys	*A large language model*
Words it knows	8	**about 100 000**

From our toys to a large language model

	Our toys	*A large language model*
Words it knows	8	**about 100 000**
Numbers in an embedding	6	**thousands**

From our toys to a large language model

	Our toys	*A large language model*
Words it knows	8	**about 100 000**
Numbers in an embedding	6	**thousands**
Numbers learned from text	14	**billions**

From our toys to a large language model

	Our toys	*A large language model*
Words it knows	8	about 100 000
Numbers in an embedding	6	thousands
Numbers learned from text	14	billions
Text it learned from	5 sentences	trillions of words

A prompt is a prefix

You

What does Alice like?

The model

Alice plays golf

A prompt is a prefix

You

What does Alice like?

The model

Alice plays golf

What does Alice like? **Alice plays golf** END

The prompt is the prefix

A prompt is a prefix

You

What does Alice like?

The model

Alice plays golf

What does Alice like? **Alice plays golf** END

The prompt is the prefix *The answer, one word at a time*

One step: one new word

Prefix

What does Alice like? Alice

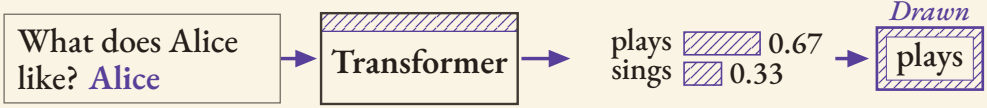
One step: one new word

Prefix



One step: one new word

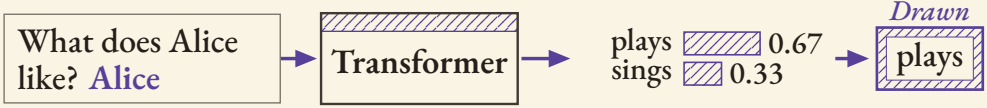
Prefix



Draw one word at random, by its chance: plays comes out two times in three.

One step: one new word

Prefix



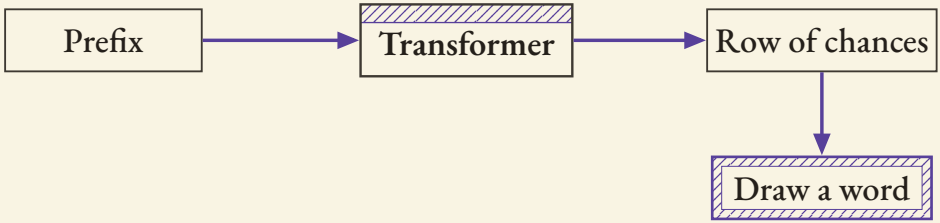
Draw one word at random, by its chance: plays comes out two times in three.

One time in three, sings comes out instead.

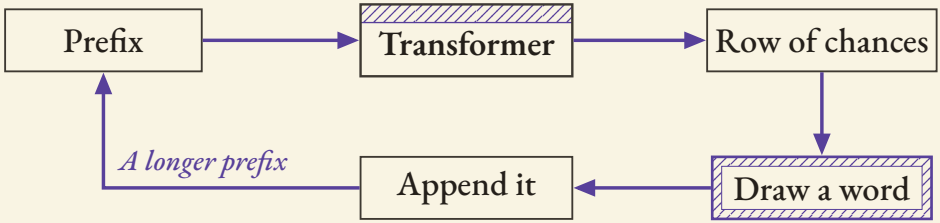
Append the word and repeat



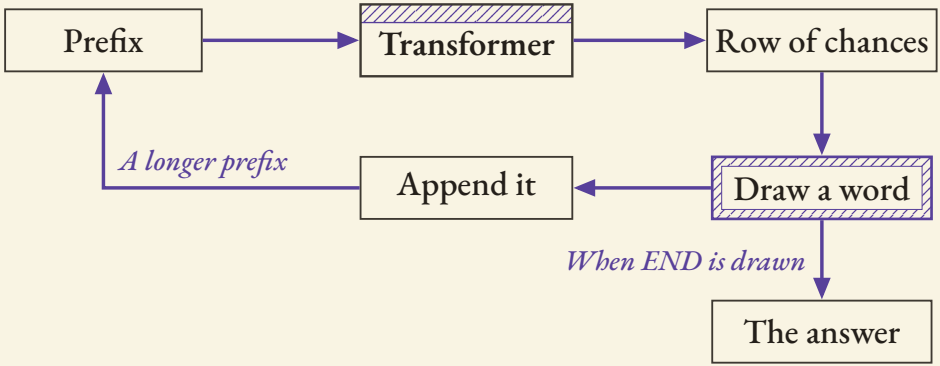
Append the word and repeat



Append the word and repeat



Append the word and repeat



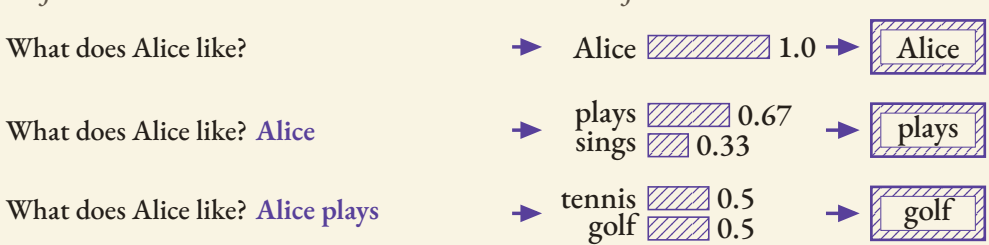
An answer, word by word



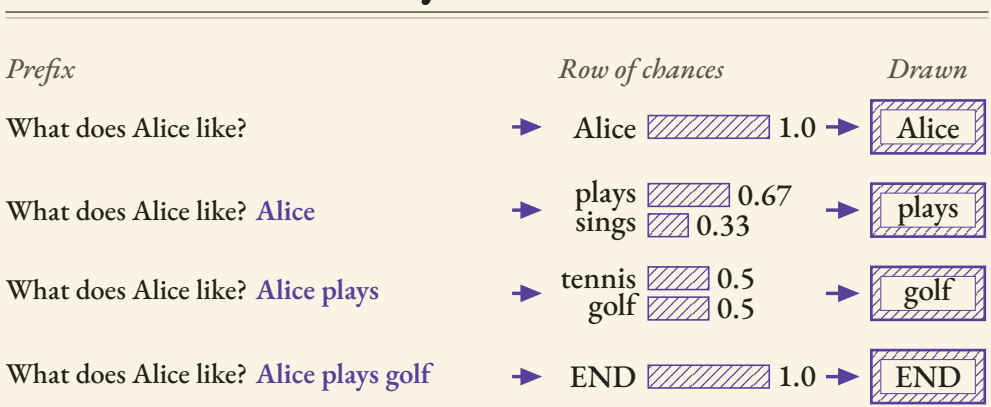
An answer, word by word



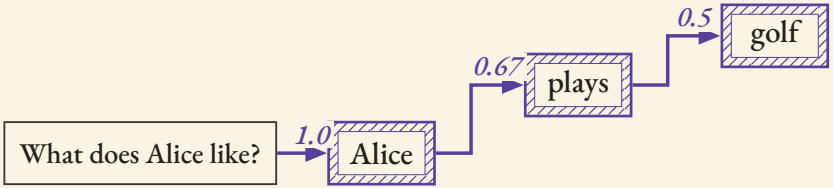
An answer, word by word



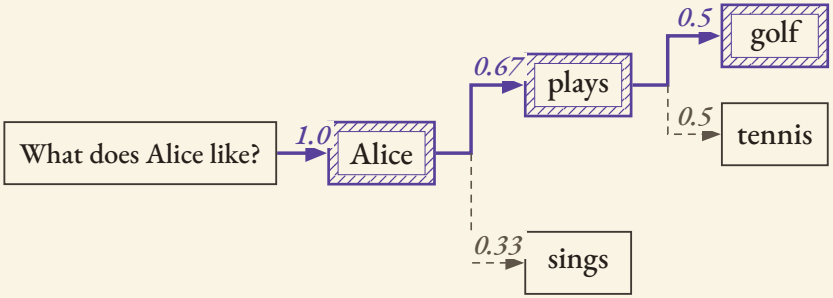
An answer, word by word



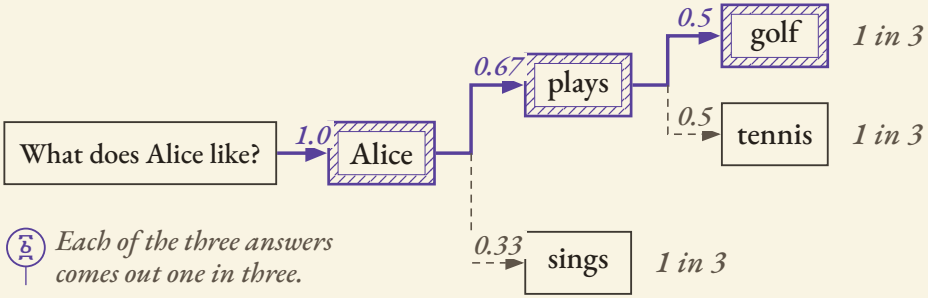
Ask again, get another answer



Ask again, get another answer

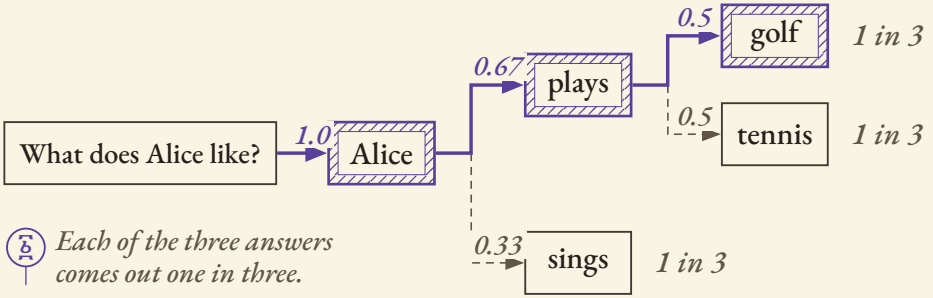


Ask again, get another answer



Each of the three answers comes out one in three.

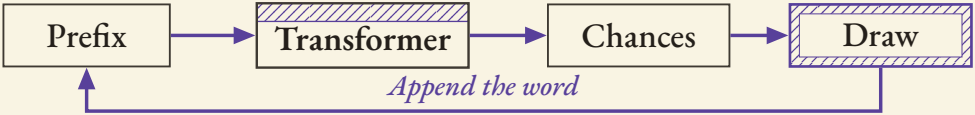
Ask again, get another answer



Each of the three answers comes out one in three.

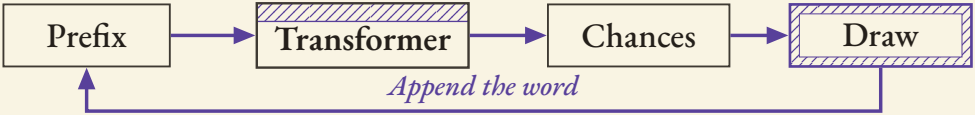
Ask again, and the answer can change.

How an LLM works



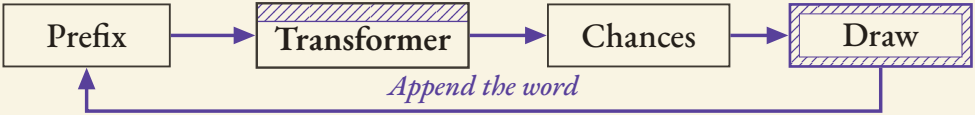
- A **prompt** is a prefix

How an LLM works



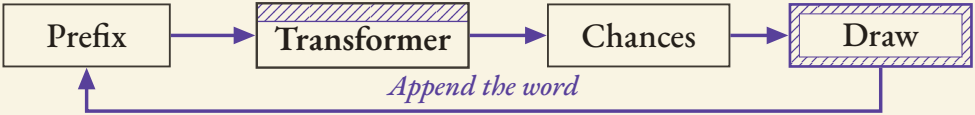
- A **prompt** is a prefix
- A **transformer** turns it into an embedding, then into a row of chances

How an LLM works



- A **prompt** is a prefix
- A **transformer** turns it into an embedding, then into a row of chances
- One word is drawn by its chance and appended to the prefix

How an LLM works



- A **prompt** is a prefix
- A **transformer** turns it into an embedding, then into a row of chances
- One word is drawn by its chance and appended to the prefix
- Repeat until END: the words drawn are the answer